What Keeps a Generative Recommender Relevant? An Empirical Study of Reinforcement Fine-Tuning Against a Learned Reward Model

Daniil Antonov
ITMO University, Saint Petersburg, Russia
Draft, version 4 18 October 2026

Abstract. A recommender that generates a list item by item can be fine-tuned with reinforcement learning (RL) towards feedback that next-item training does not see. We study the least protected form of this idea: a GPT-2 recommender is fine-tuned with a policy gradient to raise the consumed share of its items as a second GPT-2, the reward model, predicts it, held near the pretrained recommender only by a KL anchor. On a video log (KION) and a music log (Yambda-50M), with three seeds for every central comparison, instruments that the optimised model cannot influence judge the result: a reward model fitted on other users, the logged response to recommended items that users went on to consume, and the same lists scored for another user. Relevance is traded for predicted reward along a frontier: at the operating point selected on validation requests KION loses 1.8% of NDCG@10 for an 8% higher predicted share, further along it 11.5% for 24%. The policy learns which items everybody finishes rather than what each user wants. RL settings only choose a point on the frontier, the reward function moves it, and re-ranking by the same model, which computes the KL-anchored optimum in closed form, reaches more reward at equal relevance. Rewards for return and adoption buy little offline, and the largest such gain lies beyond any return rate in the log.

1 Introduction

Generative sequential recommenders [1], [2], [3], [4], [5] are trained to continue logged sequences, and the log records what users did, not what served them: 36% of music listens in Yambda stop before half of the track, and 52% of KION titles are abandoned before half of their length. A recommender that generates a list item by item is a policy, so RL can move it towards what will be received well [6], [7], [8], [9]. Where the reward comes from decides how safe this is. Petrov and Macdonald [8] keep relevance inside the reward. Matveev et al. [9] learn from logged feedback, and use a learned feedback model to evaluate and to select recommendations at inference time. Letting a learned model of user feedback *be* the training reward is the option with the fewest protections: a policy optimised against a learned model drifts to where the model is wrong [10], [11], in recommendation towards popularity and filter bubbles [12], [13].

We study this option in its barest form. A GPT-2 [14] recommender is fine-tuned to raise the consumed share that a second GPT-2 predicts, and a KL penalty towards the pretrained recommender, the *anchor*, is the only term that asks for relevance. Since every reward gain in this setting is reported by a model, the study rests on instruments the optimised model cannot influence, and on three seeds for every central comparison. It answers five questions about RL fine-tuning of a next-item recommender; Table 1 answers each in one sentence, and Section 5.1 to Section 5.5 give the evidence.

Table 1: The five questions and their answers. Numbers are from KION unless Yambda is named; "±" is the standard deviation over three seeds.

Question	Answer in one sentence
Q1. How does the model's behaviour change after RL fine-tuning?	The predicted reward is maximised (+24% on KION, confirmed by a held-out reward model), but the policy learns which titles everybody finishes rather than what each user wants: it moves impressions from long series to short films, 78% of its change is one shift per item, and users take to the recommended titles less than is usual for them, because the reward scores only items a user has started and RL extracts its per-item part.
Q2. Is relevance preserved? Does the model recommend the same items to everyone?	Relevance is traded along a frontier: at the weight selected on validation requests KION's NDCG@10 falls by 1.8% for an 8% higher predicted share, at the main run by $11.5\% \pm 1.3$ for 24% (at every cut-off and most for users with short histories, while the most shown 1% of titles take 76% of impressions instead of 72%); Yambda keeps both, and without the anchor every user receives the same list.
Q3. Can we get closer to long-term goals?	Return, likes and adoption can be predicted and rewarded, but at a moderate distance their predictions rise by a few percent (adoption: +19% at no loss of relevance, with a fifth less coverage); the largest gain, the return after the list on KION (+18%), lies beyond any return rate in the log, and only an online test can show whether users return more.
Q4. What happens when generating several steps ahead?	A generated list gains after a hit and loses after a miss (on Yambda the item at position 5 is a hit in 54% of lists after a hit at position 4, against 25% in a ranked list, and in 4% against 6% after a miss); the predicted reward of later steps depends on an assumption about responses nobody observed, which a reward model trained with hidden responses makes unbiased; crediting an item with the rest of the list helps no per-item reward.
Q5. What is the trade-off between relevance and reward?	RL settings choose a point on the frontier; only the reward function moves the frontier: within 1% of the base recommender's NDCG@10 on KION, RL on the share gains 4% of predicted share, re-ranking by the same reward model, which computes the anchored optimum, 15%, RL with logged relevance in the reward 17%, and re-ranking by the share relative to the item's usual raises NDCG@10 and a length-neutral logged response at once.

2 Related Work

Next-item models [1], [15], [16] run autoregressively as generative recommenders over items [2] or semantic identifiers [3], up to industrial scale [4], [5]. RL enters recommendation through off-policy policy gradients [6], slate values [7] and RL heads [17], and learned simulators replace the user [18], [19]. A policy optimised against a learned model exploits its errors [10], [11], [20], [21], and an ensemble of reward models does not stop this, because models trained alike share their errors [22]. Penalising divergence from a pretrained model is KL-control [23], [24], [25]; Gao et al. [10] compare best-of-n sampling with RL along this divergence. Consumption time is confounded by item duration [26], [27], retention has been optimised directly [28], [29], and maximal marginal relevance [30] greedily optimises a list-dependent reward.

Closest to this study, Chidambaram et al. [31] find that a learned reward model for a generative recommender predicts no better than item means, that PPO and DPO against it collapse ranking metrics, and that exponential reward-weighted fine-tuning on logged rewards does better; they derive the KL-anchored optimum in closed form and note that standardising rewards rescales its temperature. OneRec-V2 [32] found its reward model open to reward hacking and moved to user feedback, scoring watch time as a percentile within the user's own history of videos of similar duration. Our setting differs in what the reward model knows: it predicts responses far better than item means (AUC 0.755 against 0.656 on KION), yet RL extracts only its per-item part. We trace this before any collapse, at controlled KL distances, on two domains with three seeds, with instruments that separate what the policy learned about items from what it learned about users, and through generation several steps ahead.

3 Method

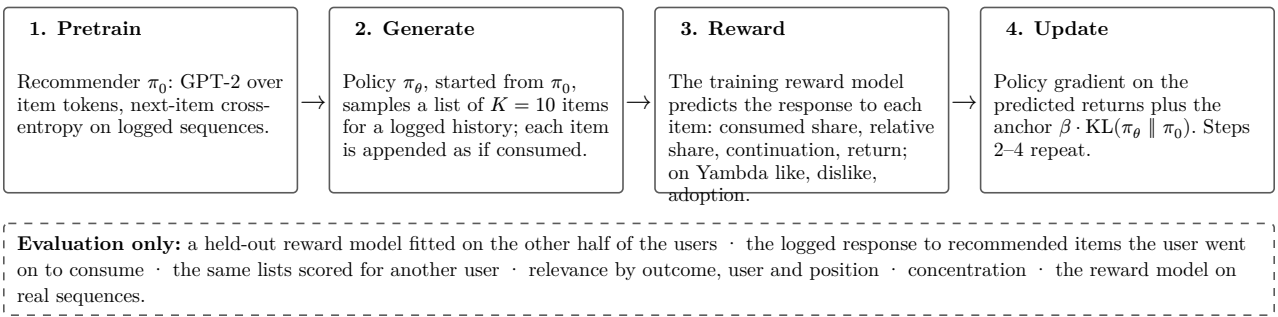


Figure 1: The pipeline. The reward model of step 3 is the only source of reward; the instruments in the dashed box never influence the policy.

Models. A log is a set of events with a user, an item, a time and the *consumed share* $\rho \in [0, 1]$ of the track or title. The recommender π_0 is a GPT-2 decoder [14] over item tokens, trained with the full-softmax next-item cross-entropy. A list of $K = 10$ is obtained by *ranking* (the catalogue is scored once) or by *generation* (each chosen item is appended to the history as if consumed) [2]; fine-tuning changes the second mode. The *reward model* is a second GPT-2 whose token at position t carries the item i_t and the consumed share of the previous item, so it scores a candidate before its outcome exists. Its heads, trained with binary cross-entropy, predict the consumed share $\hat{\rho}$ (with ρ as a soft label); the *relative share* $\hat{q}$, the share as a mid-rank quantile among the item's training-period shares, which averages one half for every item and so cannot be earned by an item that is easy to finish; the probability that the session continues, $\hat{c}$, and that the user returns, $\hat{u}$; and on Yambda the probabilities of a like and a dislike, and of *adoption* $\hat{\alpha}$: a first play followed by another play in a later session within 14 days. Since the log holds only items that users chose, every head describes the response to an item *given* that the user starts it. Two copies are trained on disjoint halves of the users: the *training* model gives the rewards, the *held-out* model only judges. A third pair is trained with *hidden responses*: in half of the training windows the responses to the last 1 to 10 events are replaced by an "unknown" value, which is the situation of a generated list.

Fine-tuning. An episode generates one list for a training history; the reward model is the environment and is fed its own expectation for the responses to earlier generated items. The policy gradient uses a learned value model with generalised advantage estimates [33] and four steps of PPO's clipped surrogate per batch [34], or a leave-one-out baseline over four lists per history [35]; advantages are standardised in both cases. The loss adds the exact divergence from π_0 at every position,

$$\mathcal{L}(\theta) = \mathcal{L}_{\mathrm{PG}}(\theta) + \beta \cdot \mathbb{E}_k[\mathrm{KL}(\pi_\theta(\cdot \mid s_k) \parallel \pi_0(\cdot \mid s_k))], \tag{1}$$

and a *KL budget* δ adapts β after every iteration so that the policy settles at δ nats per item. Two ablations remove credit along the list: $\gamma = 0$, and one-item episodes. For a reward $r(j \mid s)$ of item j in state s, the anchored objective at one position has the closed-form maximiser [23], [31]

$$\pi^*(j \mid s) = \frac{\pi_0(j \mid s) \exp(r(j \mid s)/\beta)}{Z(s)}. \tag{2}$$

Standardising the advantages makes the policy-gradient term scale-free but leaves the KL term as it is, so RL pursues this optimum with an effective weight $\beta\sigma_A$, where σ_A is the scale of the advantages before standardisation [31]. σ_A depends on the estimator and drifts during training: a weight cannot be compared between runs, the KL distance a run ends at can.

Rewards. All rewards are instances of

$$r_k = w_\rho \hat{\rho}_k + w_q \hat{q}_k + w_u \hat{u}_k + w_L\left(\hat{l}_k - \hat{d}_k\right) + w_A \hat{\alpha}_k + w_R \mathbb{1}[k = K]\hat{u}_K + w_F \frac{\mathbb{1}[a_k \in F]}{\log_2(k+1)}. \tag{3}$$

The default is $w_\rho = 1$ alone. The *return after the list* (w_R) is paid at the last item only, so it belongs to the list as a whole. The *logged-relevance* term of [8] (w_F) uses the user's real continuation F of a training history and sums to DCG@K. A *personal* reward subtracts what the model predicts for the same list shown to another user, and a *diversity* reward adds the mean distance between the items of the list.

Alternatives to on-policy RL. *Liked-only fine-tuning* continues next-item training on targets with $\rho \geq 0.5$. *Off-policy REINFORCE* [9] weights logged items by the logged reward that followed them, with importance weights capped at $\kappa = 0.5$. *Re-ranking* keeps the K of the 100 best candidates of π_0 with the highest $r(j \mid h) + w \log \pi_0(j \mid h)$, which orders items exactly as π^* of Equation 2 with $\beta = w$: it computes the top of the anchored optimum for the real history, without training.

Instruments. Let F be the next 20 items the user really consumed. A gain the *held-out reward model* does not confirm is exploitation of the training model. The *logged response* needs no model: the *logged share* $\frac{1}{K}\sum_k \mathbb{1}[a_k \in F]\rho(a_k)$ counts hits as much as responses, and the *logged surplus* $\frac{1}{K}\sum_k \mathbb{1}[a_k \in F]\big(q(a_k) - \frac{1}{2}\big)$ is positive only if users took to the recommended items more than is usual for those items. Scoring a user's lists for *another user* gives the share of a gain that depends on who receives the list, its *personal share*. *Relevance* is NDCG, Recall and MAP at $K = 1, 5, 10$ against F, and NDCG@10 or Recall@10 by outcome (liked, $\rho \geq 0.5$, or skipped; above or below the item's usual share), by the length of the history, and by position after a hit or a miss. *Concentration* is catalogue coverage, the Gini coefficient of impressions, the share of impressions of the 1% most shown items, and the overlap of two users' lists. The *per-item shift* is the π_0-weighted share of the variance of $\log \pi_\theta(j \mid h) - \log \pi_0(j \mid h)$ over histories and items that the item's mean explains: 1 for one preference per item applied to every user, and a lower bound, since the per-history normaliser is left

in. *Simulation on real sequences* runs the reward model on the user's real next five items, with the earlier responses as logged or as a simulation has to assume them.

4 Experimental Setup

Data. KION [36] is a log of an online cinema with one row per user and title, dated by its last view; we keep users with at least 5 and titles with at least 10 training events (256k users, 8k titles, 3.5M training events, a median of 10 per user), and its English item table gives each title's name and type. Yambda-50M [37] is a listening log of Yandex Music; we keep users with at least 50 training events and the 50k most played tracks (8.7k users, 35M training events, a median of 2,638 per user), with likes (0.77% of events) and dislikes (0.16%); 24% of first plays are adopted. A session continues if the next event follows within 30 minutes on Yambda and on the same calendar day on KION; the user returns if a new session starts within a day on Yambda and within a week on KION. The last 7 days of each log are the test period and the 7 days before them the validation period; the catalogue and every item statistic come from the training period, so no model sees an event after its request. The consumed share is bimodal, and on KION it depends on length: films are watched to 0.5 on average and the longest tenth of the catalogue, series, to 0.22, since the share is taken of the whole series.

Training and evaluation. All transformers have 4 layers of width 256 and windows of 48 (KION) or 128 (Yambda) events. Fine-tuning samples 512 lists per iteration ($\gamma = 0.99$, learning rate 10^{-5}) for histories cut at a random moment of the training period of a random user with at least 8 training events; the *main run* has $\beta = 0.3$ and 2,000 iterations, every other run 1,000. Items of the history are blocked on KION and allowed on Yambda, where repeats are normal. Each test user makes one request at the start of the test week: the last 38 (KION) or 118 (Yambda) events are the history, the same window that every baseline reads, and lists of ten are generated greedily unless ranked lists are named. The KL distance of a run is the exact divergence over the catalogue on its own sampled training lists, averaged over the last 50 iterations. Operating points are selected on validation requests only: the highest held-out predicted share within 5% of the base recommender's NDCG@10 selects $\beta = 1$ on KION and $\beta = 0.05$ on Yambda. The main run's $\beta = 0.3$ was carried over from earlier iterations of the study and shows a clearly visible move along the frontier, not an optimum.

Sessions and seeds. Each dataset ran a *final* session with the full grid (6.1 hours on KION, 12.3 on Yambda, on one 96 GB GPU), a *review* session with the long-term rewards, the reward models with hidden responses and the analyses by position, user and item, and two sessions with seeds 1 and 2 of the core: base models, main run, anchor sweep, a KL budget, credit and estimator comparisons, alternatives to RL and logged relevance. "±" is the standard deviation over the three seeds (Table 6); other numbers are from seed 0. The review session retrains every model with the same seed, so a run present in both sessions differs in the third decimal (the main run's coverage is 0.452 and 0.449); every table names its session. Every conclusion below holds in each seed.

5 Results

Base models. On KION the recommender is level with GRU4Rec, 13% above SASRec and 5% below BERT4Rec for ranked lists; on Yambda it is the best trained model, but below a replay of the user's own history, which reads the same 118 events (NDCG@10 0.117 against 0.187, mostly on repeated tracks), so relevance there is measured against this recommender (Table 11). The reward model predicts whether a held-out event is consumed at least half with AUC 0.755 on KION and 0.868 on Yambda, against 0.656 and 0.607 for the item's mean share; the like and dislike heads reach 0.74 and 0.73. The two copies are calibrated on their own users and differ in level on the same lists (0.417 and 0.455 for the base recommender on KION), so we compare their relative changes.

5.1 Q1. How does the model's behaviour change after RL fine-tuning?

Is the reward maximised? As far as the reward models can tell, yes: on KION the main run raises the consumed share predicted by the training reward model by 20.3% and by the held-out one by 21.5% ($23.9\% \pm 2.1$; Table 9). The log does not show users responding better. Against what is usual for each title, users took to the recommended titles less: the logged surplus falls from 0.0012 to 0.0005, by 0.0007 in every seed, and by 15% at the validation-selected $\beta = 1$. The logged share per recommended item, which mostly counts hits, rises by $1.1\% \pm 0.4$: users reached fewer recommended titles (hits −8.3%) and finished those more often (+9.8%), because they are titles everybody finishes.

Table 2: KION: the titles whose number of lists among the 59,997 test requests grew and shrank most under the main run (review session). Titles as given in the English release of the KION item table, which translates the Russian titles; "usual share": the title's mean consumed share in the training period.

	Type	Hours	Viewers in training	Usual share	Lists before	after
Gained most						
White Snow (2021)	film	2.2	32,435	0.64	8,388	24,472
Palmyra (2020)	film	1.4	18,762	0.54	8,598	21,579
Devyataev (2021)	film	1.9	108,030	0.69	14,471	26,328
Dukhless (2011)	film	1.7	953	0.55	7,308	17,315
Masha (2020)	film	1.5	40,254	0.63	23,485	32,316
Lost most						
Themis sees (2019)	series	16.9	25,571	0.08	33,948	10,781
Listened to (2021)	series	9.5	62,651	0.18	16,278	3,343
The Scam (2021)	series	22.0	51,094	0.11	16,660	4,038
The Germans (2021)	series	10.9	33,452	0.29	14,654	5,371
Flash radio (2019)	film	1.7	8,811	0.33	11,925	5,088

How the recommendations differ. Films take 66% of the impressions of the base recommender's generated lists and 80% of the main run's (89% at $\beta = 0.1$, and all of them without an anchor). The titles that gained most are films of 1.4 to 2.2 hours usually watched to 0.54–0.69; four of the five that lost most are series of 9.5 to 22 hours whose usual share is 0.08 to 0.29 (Table 2). Popularity is not what the policy follows: one of the largest gains is a film that 953 users watched in the training period.

One shift per item. On KION the item's mean change explains 78% of the change of the policy's log-probabilities, and an item's mean shift correlates 0.90 with its usual consumed share, 0.12 with its log popularity and -0.36 with its duration (2,048 test histories, review session). The policy learned a ranking of titles by how often they are finished and applies it to every user. On Yambda the item means explain 44%.

Why. There are two causes. The first is the reward: $\hat{\rho}$ is the share a user consumes given that they start the item, so it cannot penalise an item the user would never start, and on KION the share of a series is taken over all its episodes, so an item that everybody finishes earns a high reward whoever receives it. The second is the optimiser: the reward model knows something about users ($8.9\% \pm 0.3$ of the gain of re-ranking by it is personal), but a policy gradient, averaging a noisy signal over histories, accumulates first the direction that is the same for all of them. The same lists scored for another user earn the same gain (personal share $0.2\% \pm 0.5\%$), and re-ranking by the item's mean share in the training log, with no reward model, reaches the main run's point (predicted share 0.549 at NDCG@10 0.109, against 0.553 at 0.105). On Yambda the reward model knows more about the user than RL uses ($55\% \pm 9$ of re-ranking's gain is personal, $16\% \pm 16$ of RL's), and the policy changes little (predicted share $+1.67\% \pm 0.01$, NDCG@10 $+0.8\% \pm 0.7$).

> **Answer to Q1.** The reward is maximised as far as the reward models can tell, but the policy learns which items everybody finishes, not what each user wants: the reward scores only items users have started, and RL extracts the per-item part of a reward model that knows more; measured against what is usual for each title, users took to the recommendations less.

5.2 Q2. Is relevance preserved? Does the model recommend the same items to everyone?

A frontier, not a verdict. How much relevance is kept is a choice of operating point (Figure 2). At the weight selected on validation requests, $\beta = 1$ on KION, NDCG@10 falls by 1.8% for an 8.3% higher predicted share and a 3.3% higher logged share (three seeds); at the main run, $\beta = 0.3$, it falls by $11.5\% \pm 1.3$ for 24%. On Yambda the validation rule selects $\beta = 0.05$ (NDCG@10 -4.9%, predicted share $+5.0\%$), one grid step from the weight that collapses the policy in every seed; the main run moves little.

Relevance by Recall, NDCG and MAP. At the main run on KION every metric falls at every cut-off, by 8 to 14% (Table 3). The first item is chosen from the real history alone, so the loss at $K = 1$ (-7.9%) is the policy's own preference; from the second position on, the generated context adds to it (Section 5.4). By outcome, NDCG@10 on liked titles is kept ($+1.3\%$), Recall@10 on skipped titles falls by 18.2% and NDCG@10 on titles watched above their usual share by 9.4%. On Yambda every metric stays within 1.3% of the base recommender. On KION the users with the least history lose most: -13.5% of NDCG@10 for 5 to 8 earlier events against -7.4% for 25 or more (Table 7); on Yambda no group loses.

Table 3: Ranking metrics of the main run against the base recommender (generated lists, final session). At $K = 1$ NDCG and MAP coincide.

	KION			Yambda		
	before	**after**	**change, %**	**before**	**after**	**change, %**
NDCG						
NDCG@1	0.0885	0.0814	−7.9	0.1667	0.1655	−0.7
NDCG@5	0.1044	0.0920	−11.9	0.1336	0.1342	+0.4
NDCG@10	0.1173	0.1047	−10.7	0.1172	0.1183	+0.9
Recall						
Recall@1	0.0439	0.0389	−11.4	0.0159	0.0157	−1.3
Recall@5	0.1220	0.1050	−14.0	0.0507	0.0508	+0.2
Recall@10	0.1620	0.1453	−10.3	0.0747	0.0756	+1.2
MAP						
MAP@1	0.0885	0.0814	−7.9	0.1667	0.1655	−0.7
MAP@5	0.0781	0.0682	−12.6	0.0997	0.0996	−0.1
MAP@10	0.0819	0.0719	−12.2	0.0753	0.0754	+0.1

The same items for everyone? Not at the main run, but closer to it (Table 8). On KION the base recommender's generated lists already give the 1% most shown titles 72% of the impressions (62% for ranked lists); the main run raises this to 76% and the Gini coefficient of impressions from 0.963 to 0.969. With a weaker anchor the lists converge: 92% at $\beta = 0.1$, and without an anchor nearly every user receives the same list (pair overlap 0.98 on KION, 1.00 on Yambda), which the held-out model still scores 51% above the base recommender. On Yambda the share moves from 18% to 20%.

What keeps relevance. Only the anchor asks for relevance, and its weight does not describe how much it keeps. At $\beta = 0.3$ on KION the leave-one-out estimator ends 6.7 times further from π_0 than the value model (KL 0.543 ± 0.010 against 0.081 ± 0.004), while under a KL budget of 0.1 the two end at the same distance and relevance (NDCG@10 0.1037 ± 0.0021 and 0.1013 ± 0.0026) with weights 2.9 times apart. The effective weight $\beta\sigma_A$ predicts this if the value model's advantages are about three times larger before standardisation: on the value model's curve of KL against β, the leave-one-out run at $\beta = 0.3$ sits at $\beta \approx 0.09$ (a consistency check; σ_A was not logged). The KL distance describes far more than the weight but does not pin relevance down: at an equal budget, crediting each item with its own reward is 5.6% more relevant on KION and 12% on Yambda, at a lower predicted share. Every run shares the frontier of relevance against reward (Section 5.5). On Yambda $\beta = 0.1$ keeps relevance and $\beta = 0.03$ collapses the policy in every seed (NDCG@10 0.006 ± 0.004; Table 10).

Answer to Q2. Relevance is traded along a frontier: at the validation-selected weight KION loses 1.8% of NDCG@10, at the main run 11.5%, at every cut-off and most for users with short histories, while impressions concentrate further; Yambda keeps both. The anchor keeps relevance, the KL distance describes how much far better than the weight, and without the anchor every user receives the same list.

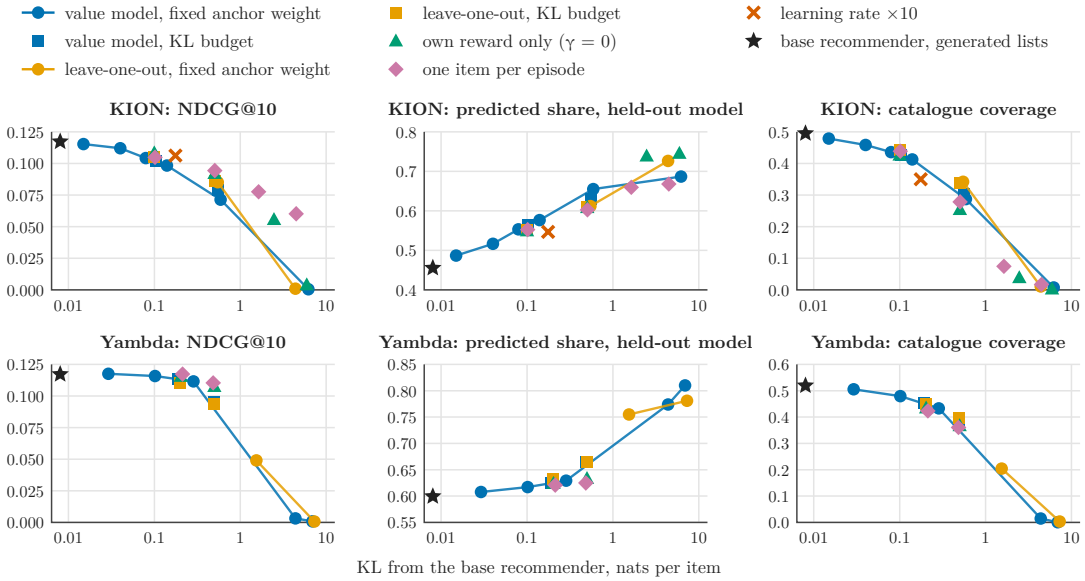


Figure 2: Every fine-tuning run of the default reward against the KL distance it ended at from the base recommender (logarithmic axis; the base recommender at the left edge; final session).

Table 4: Long-term rewards (review session). "Δ": change of NDCG@10 in percent; "Return": the held-out model's predicted return after the last item; P(like), P(adopt): its predicted probabilities (Yambda only); "Cover.": catalogue coverage; "Hours": mean duration of a recommended KION title.

	KL	NDCG@10	Δ, %	Return	P(like)	P(adopt)	Logged	Cover.	Hours
KION									
Base recommender	0.000	0.1173	+0.0	0.728	–	–	0.0173	0.495	5.7
Return after list, $\beta = 0.3$	0.507	0.0361	−69.2	0.858	–	–	0.0062	0.539	2.2
Return after list, $\beta = 0.1$	3.423	0.0012	−99.0	0.888	–	–	0.0002	0.059	1.7
same, discount 0, $\beta = 0.3$	0.006	0.1144	−2.5	0.733	–	–	0.0171	0.510	5.3
Yambda									
Base recommender	0.000	0.1172	+0.0	0.892	0.0108	0.233	0.0669	0.519	–
Like − dislike, $\beta = 0.3$	0.046	0.1148	−2.0	0.891	0.0117	0.234	0.0654	0.505	–
Like − dislike, $\beta = 0.1$	0.829	0.0841	−28.2	0.878	0.0161	0.207	0.0475	0.423	–
Adoption, $\beta = 0.3$	0.141	0.1189	+1.5	0.893	0.0121	0.277	0.0686	0.409	–
Adoption, $\beta = 0.1$	1.204	0.0737	−37.1	0.888	0.0164	0.342	0.0407	0.129	–
Return after list, $\beta = 0.3$	0.021	0.1170	−0.2	0.892	0.0107	0.233	0.0677	0.521	–

5.3 Q3. Can we get closer to long-term goals?

Which signals. Four signals in the logs describe what follows an item: the session continues, the user returns, the user likes or dislikes the item, and the user adopts it (the last two on Yambda only). A reward helps a policy only if its signal differs between the items a user could be shown. The predicted return varies between one user's candidates four times (KION) and nine times (Yambda) less than between users, and the probability of a like by only 0.003 between a user's candidates. Adoption is the exception: it varies between candidates half as much as between users (0.077 against 0.165), and it is the only signal that leans towards the items users really consumed next (AUC 0.56).

Rewards for return and continued interaction. We rewarded each signal per item, and the return also for the list as a whole, after its last item. Rewarding the return of each item raises it by 3 to 7% on KION for 8 to 23% of NDCG@10 and does not move it on Yambda (Table 15). The *return after the list* moves the KION policy further than any per-item reward (KL 0.51 at $\beta = 0.3$): the predicted return rises from 0.73 to 0.86 (+18%), as the held-out model also predicts, while NDCG@10 falls by 69% and the logged share by 64% (Table 4), and the mean duration of a recommended title falls from 5.7 to 2.2 hours. KION's format, one row per user and title dated by its last view, does not explain it: in every decile of the duration of an event's own or next title, the return rate is between 0.68 and 0.77. No group comes back at 0.86: both reward models extrapolate alike, so the held-out model cannot catch it. With discount 0 the policy hardly moves (KL 0.006): this is the one reward for which credit along the list matters. On Yambda it does not move.

Likes and adoption (Yambda). The like reward at $\beta = 0.3$ raises the predicted probability of a like from 1.08% to 1.17% for 2% of NDCG@10; at $\beta = 0.1$ likes rise by 49% and dislikes by 53%: the policy finds tracks that provoke reactions, not tracks that are liked more. Adoption is the one long-term reward that moved without a loss of relevance: at $\beta = 0.3$ its predicted probability rises from 0.233 to 0.277 (+19%), NDCG@10 by 1.5% and the logged share by 2.5%, while coverage falls from 0.52 to 0.41.

Offline or online. Offline we can check whether a signal is predictable, whether it varies between the items a policy chooses from, whether a held-out model agrees, and what it costs in relevance. Whether users return, continue or adopt more under a new policy is counterfactual: the log shows these outcomes only under the policy that produced it, adoption needs 14 days after a first play, which a test week does not have, and a prediction can leave the log's range, as the return after the list did. This needs the online experiment of Section 6.1.

> **Answer to Q3.** Rewards for return, likes and adoption can be learned and optimised, but at a moderate distance their predictions rise by a few percent; adoption is the most promising signal, the largest gain lies outside the log, and only an online test can show whether users return more.

5.4 Q4. What happens when generating several steps ahead?

A generated list appends each chosen item to the history as if the user had consumed it, so it plans several steps ahead under an assumption that is false for most items: at most 17% of the items at any position are hits.

Effect on the following recommendations. After a miss, a generated list continues from a context that did not happen and hits less often than a ranked list: 0.0273 against 0.0312 at position 5 on KION, 0.040 against 0.063 on Yambda (Figure 3). After a hit it hits more often: 0.112 against 0.093 at position 2 on KION,

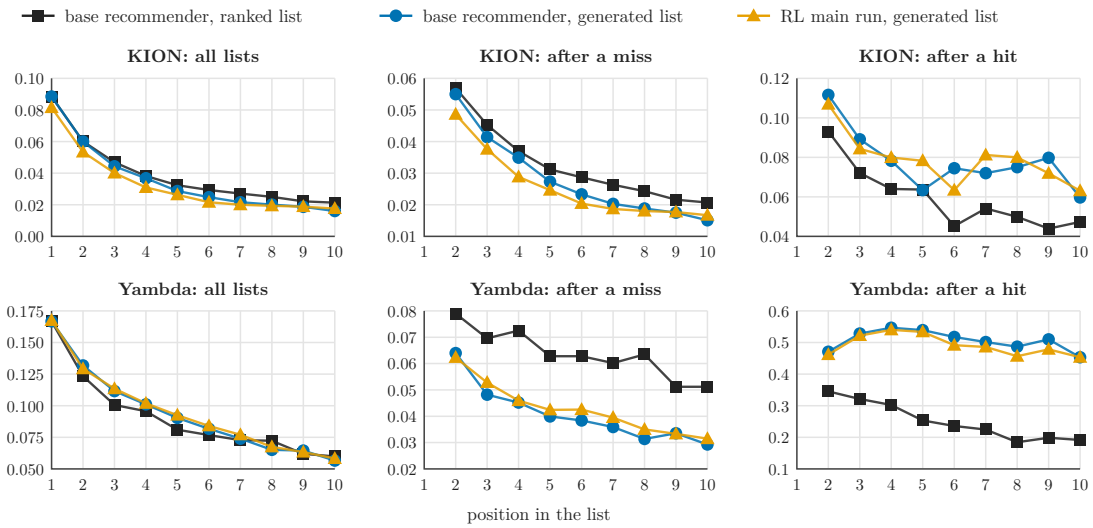


Figure 3: Hit rate by position of the list, for all lists, after a miss and after a hit at the position before: ranked and generated lists of the base recommender and generated lists of the main run (review session).

and on Yambda, where listening continues within albums and artists, 0.54 against 0.25 at position 5. Misses dominate on KION, where generation costs 4% of NDCG@10; on Yambda generation is ahead at positions 2 to 7. The fine-tuned policy keeps the pattern.

Effect on the reward estimate. On the users' real next five items, with logged responses fed back, the AUC of the predicted share rises along the list (0.757 to 0.782 on KION, 0.640 to 0.819 on Yambda), while under any simulated response it stays near its first-step level (Figure 4). The first step itself is the hardest to predict on Yambda (AUC 0.64), since test requests sit at the start of a week, after a break. Expected responses underestimate the next share on Yambda by 0.06 to 0.09, sampled ones overestimate slightly (at most +0.035), and assuming that earlier items were finished overestimates by 0.06 to 0.18. The reward model trained with hidden responses and fed "unknown" is nearly unbiased (+0.001 to +0.013) and reads observed responses as well as the standard model (AUC 0.755 against 0.758 on KION). A policy fine-tuned against it lands on the same frontier (at $\beta = 0.3$ on KION NDCG@10 0.106 against 0.104 at KL 0.078 against 0.080), as does one fine-tuned with sampled responses: the assumption sets the level of a multi-step predicted reward, not which policy wins.

Does RL use the steps? At an equal KL budget, crediting each item with its own reward only is at least as relevant as crediting it with the rest of the list: NDCG@10 0.1070 ± 0.0027 against 0.1013 ± 0.0026 on KION

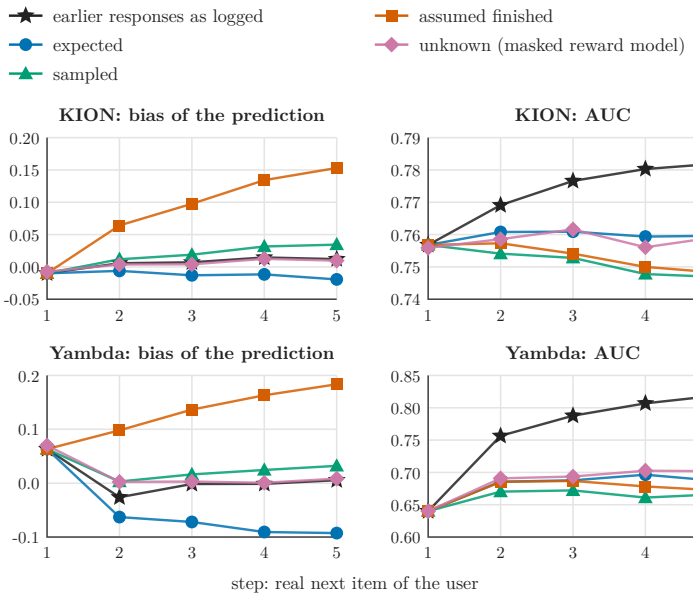


Figure 4: The reward model on the users' real next five items: bias and AUC of the predicted share at each step, with the earlier responses as logged, expected, sampled, finished, or unknown to the model trained with hidden responses (review session).

(one-item episodes 0.1043 ± 0.0024), at predicted shares 1.4 to 1.6 points lower, and 0.1076 ± 0.0014 against 0.0960 ± 0.0008 on Yambda. The myopic policies lie on the same frontier of relevance against reward. Only the return after the list, paid at the end, needed full credit (Section 5.3).

> **Answer to Q4.** Generation gains after a hit and loses after a miss; simulated responses bias the reward of later steps unless the reward model was trained to read unknown ones; and credit along the list helps no per-item reward.

5.5 Q5. What is the trade-off between relevance and reward?

How RL settings and the reward shape it. Every RL setting we varied, the anchor weight, the KL budget, the estimator, the credit assignment and the simulated responses, moves the policy along one curve (Figure 2, Section 5.4): **RL settings choose a point on the frontier; only the reward function moves the frontier.** Equation 2 says why for per-item rewards: the settings change how far the policy moves towards π^*, not where π^* lies, and only r does.

Can the reward rise without a noticeable loss of accuracy? Table 5 reads each method at equal relevance, within 1% or 5% of the base recommender's NDCG@10; between two runs of a method the curve is the straight line that a mixture of the two policies achieves. On KION, RL on the share gains 4.2% and 14.1% of predicted share and 1.8% and 3.7% of logged share. Ten times logged relevance in the reward moves the frontier: 16.9% and 23.1% of predicted share. Re-ranking by the same reward model reaches 14.9% and 20.7% with the largest logged gains (15.6%), of which its start from the ranked list contributes 11.7%. On Yambda the differences at 1% are within the noise between seeds, and the methods that learn from the log lead in logged share through repeats.

Does it take RL? Re-ranking with weight w computes the top of the anchored optimum for $\beta = w$. At the main run's predicted share it is more relevant (NDCG@10 0.1119 ± 0.0013 against 0.1041 ± 0.0020), has a higher logged share (0.0201 ± 0.0001 against 0.0176 ± 0.0002), and $8.9\% \pm 0.3$ of its gain is personal against 0.2% for RL (Table 12). Most of the relevance gap is a head start: ranked lists are 4% more relevant than generated ones on KION, and from its own start each method pays about the same relevance for the same predicted share. Re-ranking also beat RL on the diversity reward, where a greedy list is not optimal in principle (Table 13).

Rewards that move the frontier. On KION the share plus ten times logged relevance at $\beta = 0.3$ improves on the base recommender's generated lists in relevance (NDCG@10 0.1202 ± 0.0007 against 0.1177 ± 0.0008), predicted share (+10%) and logged share (+7%) at once, though not on its ranked lists (0.1221 ± 0.0004). The relative share moves the RL frontier sideways: the drift towards films disappears and relevance is kept, but almost nothing is gained (Table 14). Re-ranking by the relative share gives the one improvement of a length-neutral logged outcome in the study. Against the ranked lists it starts from, at $w = 0.3$ NDCG@10 rises from 0.1221 ± 0.0004 to 0.1234 ± 0.0006 and the logged surplus by 15%; at $w = 0.1$ the surplus rises by 38% for 0.5% of NDCG@10; both are ahead in every seed (Table 6). The personal reward cannot collapse onto one list even without an anchor (pair overlap 0.33 against 0.98 on KION) but raises the personal part only slightly

Table 5: Methods at equal relevance (final session): the largest change of the predicted share (held-out model) reached without losing more than 1% or 5% of the base recommender's NDCG@10, and the change of the logged share at that point, in percent. "–": the method does not reach that relevance.

	NDCG@10 at most 1% below the base		**NDCG@10 at most 5% below the base**	
	Predicted share, %	**Logged share, %**	**Predicted share, %**	**Logged share, %**
KION				
Ranked list of the base recommender	+2.4	+11.7	+2.4	+11.7
RL on the share, anchor sweep	+4.2	+1.8	+14.1	+3.7
RL on share + 10 × logged relevance	+16.9	+8.8	+23.1	+6.7
Re-ranking by the reward model	+14.9	+15.6	+20.7	+15.5
Off-policy REINFORCE	+1.4	+0.3	+6.9	+1.4
Liked-only fine-tuning	+1.7	+0.3	+8.4	+1.4
Yambda				
Ranked list of the base recommender	+0.6	−1.2	+1.9	−3.6
RL on the share, anchor sweep	+2.8	+0.1	+5.0	−2.5
RL on share + 10 × logged relevance	+1.4	+24.1	+1.4	+24.1
Re-ranking by the reward model	–	–	+5.3	−3.8
Off-policy REINFORCE	+1.9	+9.9	+1.9	+9.9
Liked-only fine-tuning	+1.2	+3.4	+1.2	+3.4

(Table 16). On Yambda logged relevance alone raises NDCG@10 by 23% by replaying histories: relevance on new tracks falls from 0.0260 to 0.0198, and the share of recommendations taken from the history rises from 51% to 62%.

> **Answer to Q5.** RL settings choose a point on the frontier, and only the reward function moves it. Within 1% of the base relevance, re-ranking by the same model, which computes the anchored optimum, gains more than RL; logged relevance is the reward term that raises logged outcomes; and re-ranking by the relative share improves relevance and a length-neutral logged response at once.

6 Discussion

A second reward model, the usual defence against reward hacking, catches the exploitation specific to one set of weights (0.84 against 0.69 without an anchor) and misses what any model of this kind invites, as Eisenstein et al. [22] found for ensembles: both of our models learned, correctly, that some titles are finished more often than others, and both extrapolate the return after the list past any rate in the log. Scoring the same lists for another user, splitting the policy's change into a part per item, and counting the logged response to recommended items expose this cheaply. For a reward that sums per-item predictions, the anchored optimum is a re-ranking of π_0 (Equation 2), and a policy gradient is an expensive way to approximate it; the one time sequential credit mattered, it chased a prediction outside the log. In practice: report the KL distance a run ended at, not its weight; re-rank by the same model and by item means before fine-tuning; score lists for another user; and simulate generated items with a reward model trained to read unknown responses.

6.1 An online experiment

Offline evidence settles the collapse, the role of the anchor and the impersonality of the predicted gain. Whether users consume more of the fine-tuned lists, return more often or adopt more needs an experiment that serves them. Its arms are set by KL distance, since near Yambda's cliff a fixed weight does not fix a distance.

- *Arms:* (A) the base recommender; (B) RL held at the KL distance of the validation-selected run (0.015 nats per item on KION, 0.29 on Yambda); (C) re-ranking by the reward model at the predicted share of B; (D) re-ranking by the relative share, $w = 0.3$; (E) on the music service, the adoption reward at its KL distance of 0.14.
- *Primary metrics, per user, over four weeks of hashed assignment* (one week for exposure to settle, fourteen days of adoption window, one more for returns): the consumed share of served recommendations and its surplus over the items' usual, the return rate, and on Yambda the share of first plays of recommended tracks played again within 14 days.
- *Guardrails:* consumption per user, likes and dislikes, coverage and Gini of impressions, the share of new items, and every metric for the quartile of users with the shortest histories.
- *Analysis and size:* per-user differences with a CUPED adjustment on the four weeks before, bootstrap intervals, history quartiles as pre-registered segments; a 1% relative change in a binary outcome near 0.7 needs about $2(1.96 + 0.84)^2 \cdot 0.7 \cdot 0.3/0.007^2 \approx 67,000$ users per arm ($\alpha = 0.05$, power 0.8).

7 Limitations, Future Work and Conclusion

Limitations. All results are offline, and the logged check sees only recommended items that users reached on their own. Three seeds cover the core of the study; the long-term rewards and the analyses by position, user and item have one, and adoption could not be verified on a test week. The explanation through $\beta\sigma_A$ is a consistency check, since σ_A was not logged. The models are small (4 layers of width 256), so a larger reward model may know more about users; Yambda's recommender and its replay baseline read 118 events of a median 2,638; the Yambda grids are narrower than KION's, and the baselines are untuned. Re-ranking and RL start from different lists, the equal-relevance curves interpolate between runs, and the personal part is the reward model's own belief.

Future work. The online experiment of Section 6.1 is the decisive next step. Offline, three directions follow. Rewards that no item can earn on its own: the relative share and its per-user counterpart from OneRec-V2 [32], with re-ranking by them, the one method that improved a length-neutral logged outcome. Learning from logged rewards, where the logged gains of this study came from: exponential reward-weighted fine-tuning [31] is the smooth form of our liked-only baseline. And measurement: logging σ_A and the personal share during training, splitting the reward's variance into item and user parts to predict the per-item shift before running RL, a larger reward model, a longer Yambda context, and a test period long enough to observe adoption.

Conclusion. We fine-tuned a generative recommender against a learned reward model in the setting with the fewest protections. The reward rose and a model trained on other users confirmed it, but the policy had learned which items everybody finishes, users took to the recommendations less than usual, and relevance was traded along a frontier that RL settings could only move along. Long-term rewards bought little, and the largest such gain lay outside the range of the log. What improved logged outcomes came from the log or from rewards no item can earn on its own: logged relevance in the reward, off-policy learning from logged sessions, and re-ranking by the relative share. For a reward model to be worth optimising with RL, it has to know something about the user that cannot be looked up per item: on KION that part was small for every method, on Yambda the reward model held more of it than RL used. Code, configurations and every table are in `github.com/daniilantonovv/RecSysMaxxing`.

References

[1] W.-C. Kang and J. McAuley, "Self-Attentive Sequential Recommendation," *arXiv preprint arXiv:1808.09781*, 2018.

[2] A. V. Petrov and C. Macdonald, "Generative Sequential Recommendation with GPTRec," *arXiv preprint arXiv:2306.11114*, 2023.

[3] S. Rajput *et al.*, "Recommender Systems with Generative Retrieval," *arXiv preprint arXiv:2305.05065*, 2023.

[4] J. Zhai *et al.*, "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations," *arXiv preprint arXiv:2402.17152*, 2024.

[5] J. Deng *et al.*, "OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment," *arXiv preprint arXiv:2502.18965*, 2025.

[6] M. Chen, A. Beutel, P. Covington, S. Jain, F. Belletti, and E. Chi, "Top-K Off-Policy Correction for a REINFORCE Recommender System," *arXiv preprint arXiv:1812.02353*, 2018.

[7] E. Ie *et al.*, "Reinforcement Learning for Slate-based Recommender Systems: A Tractable Decomposition and Practical Methodology," *arXiv preprint arXiv:1905.12767*, 2019.

[8] A. Petrov and C. Macdonald, "Aligning GPTRec with Beyond-Accuracy Goals with Reinforcement Learning," *arXiv preprint arXiv:2403.04875*, 2024.

[9] A. Matveev, S. Makeev, A. Krasilnikov, V. Baikalov, S. Liamaev, and K. Khrylchenko, "Session-Level Optimization for Large-Scale Retrieval using REINFORCE with Multi-Step Off-Policy Correction," *arXiv preprint arXiv:2607.02818*, 2026.

[10] L. Gao, J. Schulman, and J. Hilton, "Scaling Laws for Reward Model Overoptimization," *arXiv preprint arXiv:2210.10760*, 2022.

[11] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger, "Defining and Characterizing Reward Hacking," *arXiv preprint arXiv:2209.13085*, 2022.

[12] C. Gao *et al.*, "Alleviating Matthew Effect of Offline Reinforcement Learning in Interactive Recommendation," *arXiv preprint arXiv:2307.04571*, 2023.

[13] R. Jiang, S. Chiappa, T. Lattimore, A. György, and P. Kohli, "Degenerate Feedback Loops in Recommender Systems," *arXiv preprint arXiv:1902.10730*, 2019.

[14] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," technical report, 2019.

[15] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based Recommendations with Recurrent Neural Networks," *arXiv preprint arXiv:1511.06939*, 2015.

[16] F. Sun *et al.*, "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer," *arXiv preprint arXiv:1904.06690*, 2019.

[17] X. Xin, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Self-Supervised Reinforcement Learning for Recommender Systems," *arXiv preprint arXiv:2006.05779*, 2020.

[18] E. Ie *et al.*, "RecSim: A Configurable Simulation Platform for Recommender Systems," *arXiv preprint arXiv:1909.04847*, 2019.

[19] K. Zhao *et al.*, "KuaiSim: A Comprehensive Simulator for Recommender Systems," *arXiv preprint arXiv:2309.12645*, 2023.

[20] T. Yu *et al.*, "MOPO: Model-based Offline Policy Optimization," *arXiv preprint arXiv:2005.13239*, 2020.

[21] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, "MOReL: Model-Based Offline Reinforcement Learning," *arXiv preprint arXiv:2005.05951*, 2020.

[22] J. Eisenstein *et al.*, "Helping or Herding? Reward Model Ensembles Mitigate but do not Eliminate Reward Hacking," in *Conference on Language Modeling*, 2024.

[23] N. Jaques *et al.*, "Way Off-Policy Batch Deep Reinforcement Learning of Implicit Human Preferences in Dialog," *arXiv preprint arXiv:1907.00456*, 2019.

[24] D. M. Ziegler *et al.*, "Fine-Tuning Language Models from Human Preferences," *arXiv preprint arXiv:1909.08593*, 2019.

[25] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *arXiv preprint arXiv:2203.02155*, 2022.

[26] R. Zhan *et al.*, "Deconfounding Duration Bias in Watch-time Prediction for Video Recommendation," *arXiv preprint arXiv:2206.06003*, 2022.

[27] H. Zhao, L. Zhang, J. Xu, G. Cai, Z. Dong, and J.-R. Wen, "Uncovering User Interest from Biased and Noised Watch Time in Video Recommendation," *arXiv preprint arXiv:2308.08120*, 2023.

[28] L. Zou, L. Xia, Z. Ding, J. Song, W. Liu, and D. Yin, "Reinforcement Learning to Optimize Long-term User Engagement in Recommender Systems," *arXiv preprint arXiv:1902.05570*, 2019.

[29] Q. Cai *et al.*, "Reinforcing User Retention in a Billion Scale Short Video Recommender System," *arXiv preprint arXiv:2302.01724*, 2023.

[30] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, pp. 335–336.

[31] K. Chidambaram, S. K. Krishnamurthy, Q. Xu, K.-J. Hsiao, and M. Bhattacharya, "Robust Post-Training for Generative Recommenders: Why Exponential Reward-Weighted SFT Outperforms RLHF," *arXiv preprint arXiv:2603.10279*, 2026.

[32] G. Zhou, H. Hu, H. Cheng, H. Wang, J. Deng, and others, "OneRec-V2 Technical Report," *arXiv preprint arXiv:2508.20900*, 2025.

[33] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[35] W. Kool, H. van Hoof, and M. Welling, "Buy 4 REINFORCE Samples, Get a Baseline for Free!," in *ICLR 2019 Workshop on Deep Reinforcement Learning Meets Structured Prediction*, 2019.

[36] A. Petrov, I. Safilo, D. Tikhonovich, and D. Ignatov, "MTS Kion Implicit Contextualised Sequential Dataset for Movie Recommendation," *arXiv preprint arXiv:2209.00325*, 2022.

[37] A. Ploshkin *et al.*, "Yambda-5B — A Large-Scale Multi-modal Dataset for Ranking And Retrieval," *arXiv preprint arXiv:2505.22238*, 2025.

A Supporting tables

Table 6: The central comparisons over three seeds (0, 1, 2): mean ± standard deviation. A seed fixes the initial weights of every model and the sampling of fine-tuning. "Budget" is a KL budget of 0.1 nats on KION and 0.5 on Yambda; "–": not run with three seeds on that dataset.

	KION	**Yambda**
Main run (β = 0.3) against the base recommender		
NDCG@10, change in %	−11.5 ± 1.3	+0.8 ± 0.7
Predicted share, held-out model, change in %	+23.91 ± 2.09	+1.67 ± 0.01
Logged share per recommended item, change in %	+1.1 ± 0.4	+1.7 ± 0.7
Logged surplus per recommended item, change	−0.0007 ± 0.0000	−0.0001 ± 0.0001
Personal share of the predicted gain, %	0.2 ± 0.5	15.9 ± 16.3
KL from the base recommender	0.091 ± 0.005	0.038 ± 0.001
What the anchor weight means		
KL at $\beta = 0.3$, value model	0.081 ± 0.004	–
KL at $\beta = 0.3$, leave-one-out baseline	0.543 ± 0.010	–
Weight holding the KL budget, value model	0.256 ± 0.004	0.076 ± 0.011
Weight holding the KL budget, leave-one-out	0.752 ± 0.017	–
NDCG@10 at the budget, whole rest of the list	0.1013 ± 0.0026	0.0960 ± 0.0008
NDCG@10 at the budget, own reward only	0.1070 ± 0.0027	0.1076 ± 0.0014
NDCG@10 at the budget, one item per episode	0.1043 ± 0.0024	0.1075 ± 0.0038
NDCG@10 at $\beta = 0.03$ (Yambda's cliff)	–	0.0061 ± 0.0043
Alternatives and rewards		
NDCG@10, RL main run	0.1041 ± 0.0020	0.1190 ± 0.0015
NDCG@10, re-ranking w = 0.3	0.1119 ± 0.0013	0.1155 ± 0.0008
Logged share per item, RL main run	0.0176 ± 0.0002	0.0689 ± 0.0008
Logged share per item, re-ranking w = 0.3	0.0201 ± 0.0001	0.0662 ± 0.0007
Personal share of the gain, re-ranking w = 0.3, %	8.9 ± 0.3	54.7 ± 8.8
NDCG@10, off-policy REINFORCE	0.1035 ± 0.0006	0.1256 ± 0.0009
NDCG@10, share + 10 × logged relevance, $\beta = 0.3$	0.1202 ± 0.0007	–
NDCG@10, logged relevance alone, $\beta = 0$	0.1179 ± 0.0004	0.1453 ± 0.0014
NDCG@10 on new items, logged relevance alone	–	0.0198 ± 0.0005
Validation-selected weight on KION (β = 1)		
NDCG@10 at $\beta = 1$	0.1156 ± 0.0011	–
Predicted share at $\beta = 1$	0.478 ± 0.020	–
Logged share per item at $\beta = 1$	0.0179 ± 0.0001	–
Logged surplus per item at $\beta = 1$	0.0010 ± 0.0001	–
KL at $\beta = 1$	0.015 ± 0.001	–
Re-ranking by the relative share on KION, against the ranked lists it starts from		
NDCG@10, ranked lists	0.1221 ± 0.0004	–
NDCG@10, re-ranking w = 0.3	0.1234 ± 0.0006	–
NDCG@10, re-ranking w = 0.1	0.1214 ± 0.0019	–
Logged surplus per item, ranked lists	0.00140 ± 0.00007	–
Logged surplus per item, re-ranking w = 0.3	0.00161 ± 0.00007	–
Logged surplus per item, re-ranking w = 0.1	0.00193 ± 0.00009	–
Reference		
NDCG@10, base recommender	0.1177 ± 0.0008	0.1181 ± 0.0010
Predicted share, base recommender	0.442 ± 0.019	0.596 ± 0.005
Logged share per item, base recommender	0.0174 ± 0.0002	0.0677 ± 0.0011
Logged surplus per item, base recommender	0.0012 ± 0.0000	–
NDCG@10 on new items, base recommender	–	0.0260 ± 0.0004
Reward model AUC, held-out copy	0.754 ± 0.001	0.867 ± 0.001

Table 7: NDCG@10 and the change of the logged share per recommended item by the user's number of events before the request, in quartiles (review session, main run).

		NDCG@10				
	Users	**Base, ranked**	**Base, generated**	**RL**	**RL change, %**	**Logged change, %**
KION						
5–8 events	14552	0.1537	0.1515	0.1311	−13.5	−2.4
9–13 events	14121	0.1364	0.1311	0.1171	−10.7	+1.2
14–24 events	15849	0.1151	0.1097	0.0991	−9.7	−0.6
25–1143 events	15475	0.0861	0.0802	0.0743	−7.4	+3.3
Yambda						
50–1076 events	1897	0.1164	0.1147	0.1165	+1.6	+1.3
1077–2960 events	1899	0.1179	0.1236	0.1237	+0.1	+3.1
2961–6420 events	1898	0.1110	0.1158	0.1162	+0.3	+2.4
6421–25401 events	1899	0.1101	0.1150	0.1159	+0.8	+1.5

Table 8: Concentration of impressions: catalogue coverage, Gini coefficient of the impressions per item, and the share of impressions taken by the 1% most shown items (review session).

	Coverage	Gini of impressions	Top 1% of items, % of impressions
KION			
Base recommender, ranked	0.574	0.949	62
Base recommender, generated	0.495	0.963	72
RL, $\beta = 0.3$ (main run)	0.452	0.969	76
$\beta = 0.1$	0.278	0.990	92
$\beta = 0$	0.008	0.999	100
Yambda			
Base recommender, ranked	0.466	0.789	23
Base recommender, generated	0.519	0.737	18
RL, $\beta = 0.3$ (main run)	0.512	0.747	20
$\beta = 0.05$	0.446	0.808	29

Table 9: The main run ($\beta = 0.3$) against the base recommender, final session, seed 0. Changes are relative, in percent, with a 95% paired-bootstrap interval over users where the quantity is defined per user.

	KION			Yambda		
	before	after	change, %	before	after	change, %
Relevance against the user's logged future						
NDCG@10, all next items	0.1173	0.1047	−10.7 [−11.6, −9.8]	0.1172	0.1183	+0.9 [−0.7, +2.5]
NDCG@10, liked items	0.1214	0.1229	+1.3 [+0.1, +2.4]	0.1005	0.1029	+2.4 [+0.6, +4.3]
NDCG@10, items consumed above usual	0.1261	0.1142	−9.4 [−10.6, −8.2]	0.0998	0.1023	+2.5 [+0.7, +4.3]
Recall@10, skipped items	0.1382	0.1130	−18.2 [−19.6, −16.8]	0.0724	0.0735	+1.5 [−1.7, +4.6]
Predicted consumed share of the list						
Training reward model	0.417	0.502	+20.3 [+20.1, +20.4]	0.594	0.608	+2.2 [+2.1, +2.4]
Held-out reward model	0.455	0.553	+21.5 [+21.4, +21.7]	0.599	0.609	+1.7 [+1.5, +1.9]
Held-out, list shown to another user	0.442	0.539	+22.0 [+21.9, +22.2]	0.498	0.508	+2.0 [+1.7, +2.3]
Held-out, relative to the item's usual	0.529	0.524	−1.0 [−1.1, −1.0]	0.469	0.468	−0.3 [−0.4, −0.1]
Logged response to the recommended items the user went on to consume						
Such items per list	0.359	0.329	−8.3	0.943	0.961	+1.9
Their logged share	0.481	0.528	+9.8	0.710	0.713	+0.5
Logged share per recommended item	0.0173	0.0174	+0.6 [−0.6, +1.7]	0.0669	0.0686	+2.5 [+0.3, +4.6]
Logged share above the item's usual, per item	0.0012	0.0005	−61.9 [−69.4, −54.4]	0.0033	0.0032	−3.0 [−15.2, +9.1]
Lists						
Catalogue coverage	0.495	0.449	−9.2	0.519	0.506	−2.6
Overlap between two users' lists	0.2092	0.2509	+20.0	0.0014	0.0020	+42.9
Mean duration of an item, minutes	339.1	205.7	−39.3	3.4	3.3	−3.0

Table 10: Anchor sweep with the value-model estimator, 1,000 iterations per run (final session). KL is in nats per generated item; "Training model" is the share predicted by the model that produced the rewards.

	KL	NDCG@10	on liked	Training model	Pred. share	Logged	Coverage	Overlap
KION								
Base recommender	0.000	0.1173	0.1214	0.417	0.455	0.0173	0.495	0.209
$\beta = 1$	0.015	0.1154	0.1245	0.445	0.487	0.0178	0.479	0.219
$\beta = 0.5$	0.040	0.1120	0.1256	0.470	0.517	0.0180	0.458	0.237
$\beta = 0.3$	0.080	0.1043	0.1233	0.500	0.553	0.0175	0.436	0.251
$\beta = 0.2$	0.140	0.0983	0.1193	0.521	0.576	0.0168	0.413	0.266
$\beta = 0.1$	0.590	0.0714	0.0910	0.616	0.655	0.0114	0.287	0.316
$\beta = 0$	6.215	0.0005	0.0008	0.841	0.687	0.0002	0.008	0.983
$\beta = 0.3$, 2,000 iterations	0.092	0.1047	0.1229	0.502	0.553	0.0174	0.449	0.251
Yambda								
Base recommender	0.000	0.1172	0.1005	0.594	0.599	0.0669	0.519	0.001
$\beta = 0.3$	0.029	0.1176	0.1018	0.604	0.608	0.0681	0.505	0.002
$\beta = 0.1$	0.102	0.1158	0.1008	0.615	0.617	0.0668	0.479	0.004
$\beta = 0.05$	0.286	0.1115	0.0980	0.629	0.629	0.0653	0.433	0.006
$\beta = 0.03$	4.391	0.0032	0.0033	0.871	0.774	0.0017	0.015	0.766
$\beta = 0$	6.926	0.0008	0.0009	0.861	0.810	0.0003	0.001	0.999
$\beta = 0.3$, 2,000 iterations	0.037	0.1183	0.1029	0.608	0.609	0.0686	0.506	0.002

Table 11: Baselines on generated or ranked lists of ten (final session); every baseline reads the recommender's window of history. On KION the history's items are blocked, so replay of the history reduces to popularity and the two rows coincide. The last two rows are the recommender that is fine-tuned.

	KION			Yambda			
	NDCG@10	**Recall@10**	**Coverage**	**NDCG@10**	**on repeats**	**on new**	**Coverage**
Most popular	0.0903	0.1303	0.004	0.0186	0.0193	0.0115	0.000
Replay of the history	0.0903	0.1303	0.004	0.1873	0.3722	0.0000	0.519
Markov chain	0.0959	0.1394	0.466	0.0550	0.0934	0.0139	0.457
GRU4Rec	0.1215	0.1721	0.544	0.1116	0.1706	0.0298	0.451
SASRec	0.1082	0.1580	0.335	0.0743	0.1070	0.0242	0.367
BERT4Rec	0.1290	0.1811	0.427	0.1036	0.1717	0.0232	0.461
GPT-2, ranked	0.1220	0.1726	0.574	0.1138	0.1765	0.0315	0.466
GPT-2, generated	0.1173	0.1620	0.495	0.1172	0.1865	0.0261	0.519

Table 12: Ways of raising the reward without on-policy RL (final session). Re-ranking starts from the ranked lists of the base recommender, every other method from its generated lists.

	NDCG@10	**on liked**	**above usual**	**Pred. share**	**Logged**	**Surplus**	**Coverage**
KION							
Base recommender	0.1173	0.1214	0.1261	0.455	0.0173	0.0012	0.495
Base recommender, ranked	0.1220	0.1293	0.1326	0.466	0.0193	0.0014	0.574
RL, $\beta = 0.3$ (main run)	0.1047	0.1229	0.1142	0.553	0.0174	0.0005	0.449
Liked-only fine-tuning	0.1036	0.1258	0.1134	0.545	0.0178	0.0003	0.473
Off-policy REINFORCE, $\varkappa = 0.5$	0.1032	0.1237	0.1127	0.531	0.0179	0.0005	0.504
Re-ranking, w = 1	0.1201	0.1334	0.1315	0.497	0.0199	0.0013	0.568
Re-ranking, w = 0.3	0.1127	0.1344	0.1249	0.545	0.0201	0.0009	0.531
Re-ranking, w = 0.1	0.0953	0.1234	0.1080	0.603	0.0186	0.0003	0.434
Best of 8 sampled lists	0.1049	0.1125	0.1128	0.502	0.0178	0.0010	0.499
Yambda							
Base recommender	0.1172	0.1005	0.0998	0.599	0.0669	0.0033	0.519
Base recommender, ranked	0.1138	0.0989	0.0986	0.611	0.0645	0.0029	0.466
RL, $\beta = 0.3$ (main run)	0.1183	0.1029	0.1023	0.609	0.0686	0.0032	0.506
Liked-only fine-tuning	0.1198	0.1035	0.1028	0.607	0.0692	0.0036	0.515
Off-policy REINFORCE, $\varkappa = 0.5$	0.1245	0.1090	0.1083	0.611	0.0735	0.0039	0.475
Re-ranking, w = 1	0.1144	0.0998	0.0995	0.613	0.0651	0.0030	0.463
Re-ranking, w = 0.3	0.1155	0.1012	0.1008	0.619	0.0659	0.0029	0.456
Re-ranking, w = 0.1	0.1134	0.0998	0.0993	0.630	0.0653	0.0028	0.440
Best of 8 sampled lists	0.1110	0.0967	0.0961	0.618	0.0655	0.0030	0.498

Table 13: A reward for the list as a whole: predicted share plus the mean distance between the items of the list in the base recommender's embedding space (final session). Greedy re-ranking is maximal marginal relevance over 100 candidates.

	NDCG@10	**Pred. share**	**Distance**	**List reward**	**Coverage**
KION					
Base recommender	0.1173	0.455	0.617	0.764	0.495
RL on the list reward, $\beta = 0.3$	0.1111	0.520	0.655	0.848	0.455
RL on the list reward, $\beta = 0.1$	0.0991	0.583	0.699	0.932	0.394
Greedy re-ranking, w = 1	0.1201	0.499	0.692	0.845	0.580
Greedy re-ranking, w = 0.3	0.1104	0.550	0.749	0.925	0.571
Greedy re-ranking, w = 0.1	0.0899	0.601	0.829	1.015	0.518
Best of 8 sampled lists	0.1060	0.495	0.665	0.828	0.511
RL on the share alone, $\beta = 0.3$	0.1043	0.553	0.636	0.871	0.436
RL on the share alone, $\beta = 0.1$	0.0714	0.655	0.622	0.966	0.287
Yambda					
Base recommender	0.1172	0.599	0.715	0.957	0.519
RL on the list reward, $\beta = 0.1$	0.1182	0.611	0.732	0.976	0.509
Greedy re-ranking, w = 1	0.1145	0.613	0.744	0.984	0.465
Greedy re-ranking, w = 0.3	0.1155	0.617	0.769	1.002	0.462
Greedy re-ranking, w = 0.1	0.1105	0.624	0.821	1.034	0.447
Best of 8 sampled lists	0.1147	0.611	0.761	0.991	0.516
RL on the share alone, $\beta = 0.3$	0.1176	0.608	0.706	0.961	0.505
RL on the share alone, $\beta = 0.1$	0.1158	0.617	0.694	0.964	0.479

Table 14: Rewards that an item cannot earn on its own account, and logged relevance (final session). "Rel. share" is the held-out model's predicted share relative to the item's usual; 0.5 is "as usual".

	NDCG@10	on liked	Pred. share	Rel. share	Logged	Surplus	Coverage
KION							
Base recommender	0.1173	0.1214	0.455	0.529	0.0173	0.0012	0.495
Share (the main reward), $\beta = 0.3$	0.1043	0.1233	0.553	0.525	0.0175	0.0005	0.436
Base recommender, ranked	0.1220	0.1293	0.466	0.528	0.0193	0.0014	0.574
Relative share, $\beta = 0.3$	0.1168	0.1190	0.434	0.541	0.0168	0.0013	0.481
Relative share, $\beta = 0.1$	0.1114	0.1111	0.401	0.554	0.0154	0.0014	0.439
Re-ranking by relative share, $w = 0.1$	0.1206	0.1263	0.437	0.546	0.0185	0.0020	0.552
Share + 10 × logged relevance, $\beta = 0.3$	0.1194	0.1303	0.495	0.535	0.0185	0.0011	0.426
Share + 10 × logged relevance, $\beta = 0.1$	0.1158	0.1323	0.536	0.540	0.0188	0.0008	0.345
Logged relevance alone, $\beta = 0$	0.1180	0.1279	0.470	0.553	0.0172	0.0007	0.097
Yambda							
Base recommender	0.1172	0.1005	0.599	0.469	0.0669	0.0033	0.519
Share (the main reward), $\beta = 0.1$	0.1158	0.1008	0.617	0.465	0.0668	0.0029	0.479
Base recommender, ranked	0.1138	0.0989	0.611	0.473	0.0645	0.0029	0.466
Relative share, $\beta = 0.1$	0.1107	0.0960	0.595	0.487	0.0628	0.0040	0.476
Re-ranking by relative share, $w = 0.1$	0.1120	0.0977	0.610	0.486	0.0633	0.0034	0.444
Share + 10 × logged relevance, $\beta = 0.1$	0.1309	0.1139	0.606	0.468	0.0758	0.0035	0.516
Share + 10 × logged relevance, $\beta = 0$	0.1433	0.1245	0.608	0.467	0.0830	0.0037	0.501
Logged relevance alone, $\beta = 0$	0.1437	0.1259	0.604	0.468	0.0836	0.0039	0.486

Table 15: Rewards for continuing the session and for returning, per item (final session). "Continues" and "returns" are the held-out model's predictions; on KION a session is a calendar day.

	NDCG@10	Pred. share	Continues	Returns	Coverage	Overlap
KION						
Base recommender	0.1173	0.455	0.345	0.718	0.495	0.209
Session, $\beta = 0.3$	0.0951	0.438	0.404	0.754	0.567	0.078
Session, $\beta = 0.1$	0.0443	0.297	0.545	0.764	0.526	0.106
Return, $\beta = 0.3$	0.1077	0.459	0.368	0.739	0.521	0.141
Return, $\beta = 0.1$	0.0903	0.466	0.392	0.765	0.544	0.093
Share + return, $\beta = 0.3$	0.1091	0.502	0.355	0.733	0.491	0.183
Share + return, $\beta = 0.1$	0.0943	0.546	0.372	0.756	0.477	0.167
Yambda						
Base recommender	0.1172	0.599	0.947	0.913	0.519	0.001
Return, $\beta = 0.1$	0.1158	0.600	0.947	0.913	0.516	0.002

Table 16: Rewards with a baseline (final session): the share minus that predicted for the same list shown to another user, or minus the item's mean share. "Personal part" is the predicted share for the user minus that for another user.

	KL	NDCG@10	Pred. share	Personal part	Coverage	Overlap
KION						
Base recommender	0.000	0.1173	0.455	0.014	0.495	0.209
Personal share, $\beta = 0.3$	0.022	0.1161	0.455	0.019	0.487	0.205
Personal share, $\beta = 0.1$	0.097	0.1135	0.448	0.024	0.453	0.213
Personal share, $\beta = 0$	1.168	0.0686	0.313	0.029	0.336	0.332
Share above the item mean, $\beta = 0.3$	0.187	0.1066	0.407	0.021	0.406	0.225
Share above the item mean, $\beta = 0.1$	0.980	0.0613	0.368	0.026	0.239	0.262
Share above the item mean, $\beta = 0$	4.183	0.0327	0.424	0.002	0.038	0.797
Yambda						
Base recommender	0.000	0.1172	0.599	0.101	0.519	0.001
Personal share, $\beta = 0.1$	0.054	0.1166	0.603	0.106	0.516	0.002
Share above the item mean, $\beta = 0.1$	0.315	0.1056	0.590	0.109	0.437	0.002